

ESS PORTAL

Tech Stack Requirements:

- **Frontend:** Next.js (App Router), React, Tailwind CSS
- **Backend:** Node.js with Express
- **Database:** PostgreSQL (using an ORM like Prisma or raw SQL queries)
- **Authentication:** JSON Web Tokens (JWT) for secure role-based session management

Database Schema Setup:

Generate a migration or setup script for PostgreSQL with the following three essential tables:

1. 'Employees' Table: id, name, email, password_hash, role (ENUM: 'Employee', 'Manager'), department_id, leave_balance (default 21).
2. 'Attendance' Table: id, employee_id, date, clock_in_time, clock_out_time, total_hours.
3. 'Leaves' Table: id, employee_id, leave_type, start_date, end_date, status (ENUM: 'Pending', 'Approved', 'Rejected'), approved_by (manager_id).

Backend API Implementation (Node.js/Express):

- Build a secure, modular REST API with the following endpoints protected by JWT role verification:
- POST /api/auth/login -> Validates user credentials and returns a JWT token containing user ID and role.
- POST /api/attendance/clock-in -> Records the current timestamp for the logged-in employee.
- POST /api/attendance/clock-out -> Records the clock-out timestamp and calculates total_hours for the day.
- GET /api/attendance/summary -> Retrieves attendance history for the logged in user.

- POST /api/leaves/apply -> Allows an employee to submit a leave request specifying start_date, end_date, and leave_type. Deducts balance only upon approval.
- GET /api/leaves/pending -> (Manager Only) Fetches all pending leave requests submitted by employees.
- PUT /api/leaves/respond/:id -> (Manager Only) Updates a leave status to 'Approved' or 'Rejected'.

Frontend Interface Implementation (Next.js & Tailwind CSS):

Create a clean, scannable, mobile-responsive dashboard containing three main views based on role routing:

1. Login Page: A secure form managing JWT token assignment and storage.

2. Employee Dashboard View:

- A Quick Actions component with a prominent "Clock In / Clock Out" toggle button.
- A simple form to "Apply for Leave" (date pickers and dropdown for type).
 - Summary cards displaying current Leave Balance and a log of past Attendance hours.

3. Manager Approval View (Conditional Route):

- A dedicated dashboard section visible only to accounts with the 'Manager' role.
- A clean UI table displaying pending leave requests with immediate "Approve" and "Reject" action buttons that trigger API updates and refresh the view.

Internship Management, Collaboration & Mentorship Portal

Tech Stack Requirements:

Frontend: Next.js (App Router), React, TypeScript, Tailwind CSS

Backend: Node.js with Express (Modular Routing) - Database: PostgreSQL (using Prisma ORM or clean SQL queries)

Authentication: JSON Web Tokens (JWT) for secure role-based session management

Database Schema Setup:

Generate a migration or setup script for PostgreSQL with the following three core tables:

- 'Users' Table:** id, name, email, password_hash, role (ENUM: 'Intern', 'Mentor'), mentor_id (self-referential foreign key to link an Intern to their assigned Mentor).
- 'Tasks' Table:** id, title, description, deadline, priority (ENUM: 'Low', 'Medium', 'High'), status (ENUM: 'To-Do', 'In-Progress', 'Under-Review', 'Completed'), assigned_to (intern_id), created_by (mentor_id).
- 'Submissions' Table:** id, task_id, intern_id, submission_text, repository_url, feedback_text, rating (1-5), submission_date.

Backend API Implementation (Node.js/Express):

Build a secure, modular REST API with the following endpoints protected by JWT role-based verification:

- POST /api/auth/login -> Validates user credentials and returns a JWT token containing user ID, name, and role.

- GET /api/tasks -> Returns assigned tasks for an Intern, or all tasks created/assigned by a Mentor.
- POST /api/tasks/create -> (Mentor Only) Compiles and assigns a new task to a specific intern with a deadline.
- PUT /api/tasks/:id/status -> Allows an Intern to update task status (e.g., moving from 'In-Progress' to 'Under-Review').
- POST /api/submissions/submit -> Allows an Intern to append text or repository links to a specific task for evaluation.
- PUT /api/submissions/:id/review -> (Mentor Only) Updates the submission record with feedback text and a numerical rating.

Frontend Interface Implementation (Next.js & Tailwind CSS):

Create a clean, highly scannable UI layout containing two primary role-routed dashboards:

1. Intern Dashboard View:

- Summary metric cards: Total Assigned Tasks, Completed Tasks, and name of their assigned Mentor.
- A Kanban-inspired list or clean card grid showing assigned tasks categorized by status.
- A modal interface to submit a completed task with text inputs and a link submission field.

2. Mentor Dashboard View:

- A dropdown selector to view performance/task status filtered by individual assigned interns.

- An intuitive management form to quickly create and assign a task to a designated intern.

- A dedicated "Pending Reviews" feed displaying incoming intern submissions with text fields to write inline feedback, drop a 1-5 rating, and click a primary button to finalize the review.

Video Communication & Virtual Collaboration Platform Documentation

Tech Stack Requirements:

- **Frontend:** Next.js (App Router), TypeScript, Tailwind CSS
- **Backend:** Node.js with Express
- **Real-Time Signaling:** Socket.IO (integrated directly with the Node.js server)
- **Database:** PostgreSQL (using an ORM like Prisma for lightweight session and room logging)

Database Schema Setup:

Generate a clean, lightweight schema using Prisma or raw SQL containing two tables:

1. **'Rooms' Table:** id (UUID), room_slug (unique string for shareable URLs), created_at, is_active (boolean).
2. **'CallLogs' Table:** id, room_id, peer_id, joined_at, left_at (for monitoring raw connection lifecycles).

Backend Signaling Server Implementation (Node.js & Socket.IO):

Build a modular signaling server to broker connection handshakes without carrying heavy states:

Implement a basic REST endpoint `POST /api/rooms/create` that generates a new unique `room_slug`.

Establish Socket.IO event handlers for real-time WebRTC negotiation:

- `join-room`: Adds a socket connection to a specific `room_slug` channel and alerts existing peers.

- ``offer``: Relays the WebRTC Session Description Protocol (SDP) offer from the caller to the receiving peer.
- ``answer``: Relays the WebRTC SDP response back to the caller.
- ``ice-candidate``: Instantly routes Network Connectivity (ICE) candidates between connected peers.
- ``send-message``: Broadcasts an in-room text chat payload to everyone active in that specific socket channel.

Frontend Real-Time Video Interface (Next.js, WebRTC APIs, Tailwind CSS):

Create a secure, clean, and highly scannable application layout consisting of:

1. Landing View: A simple interface containing a "Create Instant Meeting" action button and an "Enter Room Link" text entry box.

2. Active Meeting Room View (``/room/[room_slug]``):

- A dual-grid responsive video panel rendering the local user's camera stream (``navigator.mediaDevices.getUserMedia``) and the incoming remote peer's stream cleanly side-by-side.
- An intuitive Media Control Bar featuring visual Toggle Buttons to instantly mute/unmute audio tracks and enable/disable video tracks.
- A right-hand Floating Sidebar containing an active text chat box connected to the Socket.IO room channel, allowing users to send text messages and resource links that append instantly to a live feed.

Enterprise Work Management & Collaboration Platform

Tech Stack Requirements:

Frontend: Next.js (App Router), React, TypeScript, Tailwind CSS

Backend: Node.js with Express (Modular REST API structure)

Database: PostgreSQL (using an ORM like Prisma or raw SQL queries)

Authentication: JSON Web Tokens (JWT) for secure role-based session management

Database Schema Setup:

Generate a migration or setup script for PostgreSQL establishing the following core tables:

- 'Users' Table:** id, name, email, password_hash, role (ENUM: 'Admin', 'Member').
- 'Workspaces' Table:** id, name, created_by (user_id).
- 'Projects' Table:** id, workspace_id, name, description.
- 'Tasks' Table:** id, project_id, title, description, due_date, priority (ENUM: 'Low', 'Medium', 'High'), status (ENUM: 'To-Do', 'In-Progress', 'Done'), assigned_to (user_id).
- 'Comments' Table:** id, task_id, user_id, comment_text, created_at.

Backend API Implementation (Node.js/Express):

Build a clean, robust REST API featuring JWT authentication and workspace route filtering:

- POST /api/auth/login -> Validates user credentials and issues a secure session token.

- GET /api/workspaces -> Fetches all workspaces associated with the user's organization.
- GET /api/projects?workspaceId=X -> Retrieves projects within a targeted workspace.
- GET /api/tasks?projectId=Y -> Fetches all tasks assigned to a specific project.
- POST /api/tasks/create -> Appends a new task card with set priorities and user assignments.
- PUT /api/tasks/:id/status -> Updates a card's status column (vital for Kanban drag-and-drop actions).
- POST /api/tasks/:id/comments -> Appends collaborative feedback to a targeted task card.

Frontend Interface Implementation (Next.js & Tailwind CSS):

Design a clean, scannable workspace dashboard containing:

1. Workspace Sidebar: A simple component listing available Workspaces and Projects for fast navigation switching.

2. Interlocking Kanban Board View:

- A standard 3-column layout columns ('To-Do', 'In-Progress', 'Done') mapping out tasks as compact, clean cards showing titles, priority badges, and the assigned user's name.
- An intuitive "Add Task" button at the head of the columns to quickly open a creation form.
- An interactive action handler updating task status via a dropdown selector or basic drag-and-drop mechanics to immediately execute the status update API.

3. Task Detail & Activity Panel:

- Clicking a task card pulls up a clean detailed modal containing description fields, a due-date indicator, and a rolling vertical feed of user comments with an input box to instantly post updates.

Microsoft Teams Replica

Tech Stack Requirements:

- **Frontend:** Next.js (App Router), React, TypeScript, Tailwind CSS
- **Backend:** Node.js with Express (Modular API structure)
- **Real-Time Layer:** Socket.IO (integrated with the Node server)
- **Database:** MongoDB (using Mongoose ODM for flexible message payloads)
- **File Storage:** Mocked AWS S3 local simulation endpoint

Database Schema Setup:

Generate clean, performant Mongoose schemas for MongoDB:

1. **'User' Schema:** id, name, email, passwordHash, presenceStatus (ENUM: 'Online', 'Away', 'Busy', 'Offline'), assignedTeams (Array of IDs).
2. **'Team' Schema:** id, name, description, members (Array of User IDs).
3. **'Channel' Schema:** id, teamId, name, type (ENUM: 'Standard', 'Private').
4. **'Message' Schema:** id, senderId, recipientType (ENUM: 'Direct', 'Channel'), recipientId (User ID or Channel ID), textContent, fileUrl, createdAt.

Backend Real-Time Signaling Server (Node.js & Socket.IO):

Build a modular backend handling secure JWT authentication and state state preservation:

- Establish REST endpoints for operational queries: `GET /api/channels/:id/messages` and `POST /api/messages/upload-file`.
- Configure dedicated Socket.IO event handlers:
 - `register-user`: Maps an authenticated user ID directly to their active Socket session.

- ``presence-change``: Broadcasts a user's updated status ('Online', 'DND') instantly to their active teams.
- ``join-channel-room``: Places a user's socket connection into a specific Channel broadcast room.
- ``chat-message``: Validates, saves to MongoDB, and instantly dispatches the message block to the recipient's personal socket room or active channel channel.

Frontend Workspace Layout (Next.js & Tailwind CSS):

Design a clean, scannable multi-pane workspace layout:

1. **Primary Navigation Sidebar:** Lists the current user's details with a Presence Status Dropdown alongside an organized list of assigned Teams and corresponding Channels.
2. **Direct Message Sidebar:** Shows a historical list of recent 1-to-1 individual text threads.
3. **Active Chat Window Component:**
 - A scrollable vertical feed displaying historic and streaming messages with clear sender names, date stamps, presence status lights, and readable file attachment badges.
 - A text input bar at the foot of the screen containing an attachment upload interface and an entry input box that captures 'Enter' keystrokes to fire off data over Socket.IO seamlessly.